

Annotation de Corpus

*Annotation de corpus.
Articulation entre : tâche de TAL , corpus (documents)
et modèles d'annotation*

P. Paroubek

LISN (Laboratoire Interdisciplinaire des Sciences du Numérique) - CNRS - U. Paris-Saclay,
Campus universitaire bât 507, Rue du Belvédère, F - 91400 Orsay,
email:patrick.paroubek@lisn.upsaclay.fr

EcAuTAL25 - École d'Automne en TAL - ED STIC

LISN 31 octobre 2025 - 13h-16hh

Syntactic Annotations

Main representations

Let's have a look at the main syntactic representations and annotation formats. Syntactic analysis is important to consider in conjunction with any other kind of annotation as it represents the laws that govern the structure of the sentence independently of its meaning (but locally constrained by its meaning).

Syntactic Annotations

Main representations

Le but de l'analyse syntaxique automatique est de fournir une analyse complète ou partielle de la structure d'un énoncé en termes de :

- ▶ “**chunks**” (lit. morceaux), des séquences de mots ayant une signification/fonction syntaxique commune,
- ▶ **constituants**, des séquences de mots qui représentent une unité fonctionnelle au sein d'une structure hiérarchique,
- ▶ **dépendances**, des relations liant un mot donné (appelé “**la tête**”) et un de ses dépendants,
- ▶ ou tout autre représentation propre à la théorie syntaxique utilisée...

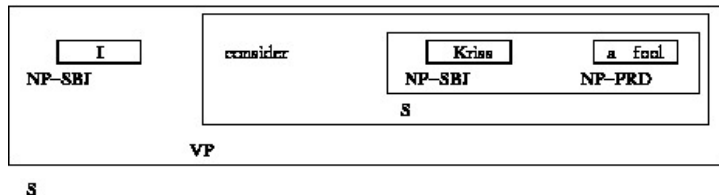
Syntactic Annotations

Main representations

Un exemple d'analyse en constituants extrait du corpus arboré (corpus avec des annotations représentant des arbres syntaxiques) PennTreebank pour l'anglais, pour l'énoncé :

"I consider Kriss a fool".

- ▶ S : clause déclarative simple (pas introduite par une conjonction de subordination ni par un pronom interrogatif et qui n'a pas d'inversion sujet-verbe)
- ▶ NP-SBJ : constituant nominal sujet de surface (sujet syntaxique),
- ▶ VP : constituant verbal
- ▶ NP-PRD : constituant nominal prédicatif

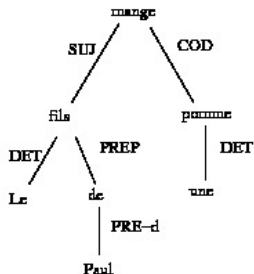
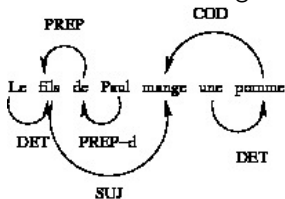


Syntactic Annotations

Main representations

Un exemple d'analyse en dépendances avec les annotations de l'analyseur SYNTAX (D. Bourrigault) :

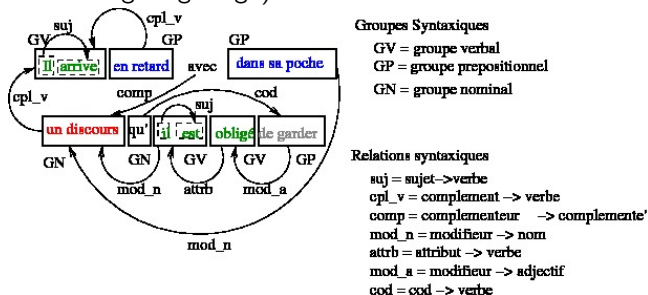
“Le fils de Paul mange une pomme”.



Syntactic Annotations

Main representations

Le formalisme de la campagne d'évaluation PASSAGE des analyseurs syntaxiques du français (2006-2009) associe dans sa représentation un niveau "chunks" (groupes syntaxiques) et des relations grammaticales (triplets ou quadruplet de la forme relation-arg1-arg2 ou relation-arg1-arg2-arg3).



Syntactic Annotations

Main representations

the PEAS annotation guidelines used for syntactic analysis of French in the PASSAGE campaign :
https://perso.limsi.fr/pap/nlp_fall_school/PEAS_reference_annotations_v2.2.html

Syntactic Annotations

Main representations

ConLL est un format de donnée extensible initialement développé pour des campagnes d'évaluation d'analyse syntaxique en dépendances et qui est utilisé par une large communauté.

Il permet de représenter les mots, d'un énoncé, les informations morpho-syntaxiques (parties du discours, lemme, etc.) et des dépendances syntaxiques. Il utilise une représentation matricielle dont la première colonne contient les formes de l'énoncé, puis les autres colonnes suivent leurs étiquettes morpho-syntaxiques, et ensuite les dépendances syntaxiques.

C'est un format extensible ; auquel on peut ajouter de nouvelles couches d'analyse par simple ajout de colonnes à la matrice de représentation. Les dépendances sont représentées au moyen de deux colonnes, l'une pour le type de la dépendance, l'autre pour l'adresse de sa cible, qui référence une ligne de la matrice, la source de la dépendance étant la forme courante.

Syntactic Annotations

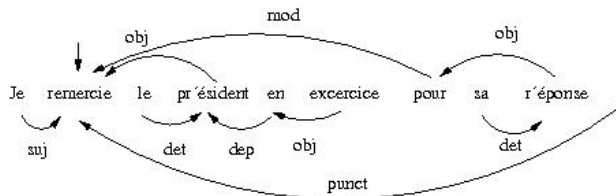
Main representations

1	Je	cln	CL	CLS	s=suj	2	subj	2	subj
2	remercie	remercier	V	V	m=ind—n=s—p=3	0	root	0	root
3	le	le	D	DET	g=m—n=s—s=def	4	det	4	det
4	président	président	N	NC	g=m—n=s—s=c	2	obj	2	obj
5	en	en	P	P	p=3	4	dep	4	dep
6	exercice	exercice	N	NC	g=m—n=s—s=c	5	obj	5	obj
7	pour	pour	P	P	-	2	mod	2	mod
8	sa	son	D	DET	g=f—n=s—s=poss	9	det	9	det
9	réponse	réponse	N	NC	g=f—n=s—s=c	7	obj	7	obj
10	.	.	PONCT	PONCT	s=s	2	ponct	2	ponct

o Extrait d'annotation ConLL issu du corpus Sequoia v4.0.

Syntactic Annotations

Main representations



Représentation graphique des dépendances de l'extrait d'annotation ConLL issu du corpus Sequoia v4.0.

Syntactic Annotations

Main representations

Ce format général et extensible (ajout de colonnes) permet de représenter une grande variété de formalisme à base de dépendances syntactiques, dont la sémantique doit être précisée dans un guide d'annotation.

Dans le cas qui nous intéresse, il s'agit du Guide d'annotation ConLL FTB :

<http://alpage.inria.fr/statgram/frdep/Publications/FTB-GuideDepSurface.pdf>.

Nous aurons dans les annotations essentiellement deux types d'information, des étiquettes (morpho-syntaxique, sémantique, se rapportant à un système de classification particulier, etc.) associées à une forme particulière, et des dépendances étiquetées qui vont relier des formes ensemble, avec la condition qu'une forme ne peut engendrer qu'une dépendance, mais par contre plusieurs dépendances peuvent aboutir sur une même forme.

Le guide d'annotation Bonsai recense 12 dépendances pour les gouverneurs verbaux :

subj (Sujet)

obj (objet)

de_obj (SP argumental en de, non locatif)

a_obj (SP argumental en à, non locatif)

p_obj (autre SP argumental)

ats (Attribut du sujet)

ato (Attribut de l'objet)

mod (Modifieur)

aux_tps (auxiliaires de temps)

aux_pass (auxiliaires du passif)

aux_caus (verbe causatif,
en cas de complexe
causatif + inf)

aff (clitiques figés)

et 8 dépendances pour gouverneurs non verbaux :

mod (Modifieurs repérés structuralement, comme par exemple les adjectifs épithétiques)

mod_rel (Relatives adnominales)

coord (Relation portée par un coordonnant, avec comme gouverneur le coordonnant)

arg (utilisé dans le cas de prépositions « liées », ex. « Charybde en Scylla »)

dep_coord (Relation portée par un coordonné différent du premier, avec comme gouverneur le coordonnant immédiatement précédent)

det (Relation portée par les déterminants)

ponct (Relation portée par tout dépendant typographique, sauf pour les virgules jouant le rôle de coordonnant)

dep (Relation sous-spécifiée, pour les dépendants prépositionnels (pas de gestion de la distinction argument / ajout pour les gouverneurs non verbaux))

Il s'agit là des représentations utilisées pour l'annotations automatique.
L'annotation manuelle du guide cité précédemment¹ ajoute les 8 dépendances suivantes :

mod_loc (Modifieurs sémantiquement locatifs, au propre ou au figuré)

mod_cleft (Pour la subordonnée dans le cas d'une clivée²)

p_obj_agt (Pour les compléments d'agent, passif ou causatif)

p_obj_loc (Dépendants argumentaux locatifs, source, destination, ou localisation)

suj_impers (Pour le sujet explétif il)

aff_moyen (Pour le clitique se en cas de moyen)

arg_comp (Utilisé pour relier une comparative et son gouverneur)

arg_cons (Utilisé pour relier une consécutive et son gouverneur adverbial)

1. Marie Candito, Benoît Crabbé, Mathieu Falco, *Dépendances syntaxiques de surface pour le français, Schéma d'annotation pour un corpus en dépendances obtenu par conversion du FrenchTreebank*

2. Les phrases clivées sont découpées en deux éléments, entre lequel un élément sur lequel on focalise l'attention est introduit, par ex. *c'est ma soeur qui a acheté une maison*

Syntactic Annotations

Différent types of analysis/representation

On distingue d'une part

- ▶ les analyses de surface (*shallow parses*), se limitant à un niveau de groupes ou de relations
- ▶ les analyses profondes (*deep parses*), construisant des structures syntaxiques sur plusieurs niveaux (récursivité)

et d'autre part

- ▶ les analyses complètes, l'énoncé est analysé entièrement (la structure syntaxique couvre tous les mots de l'énoncé et forme une structure monolithique)
- ▶ les analyses partielles, certains mots de l'énoncé sont dépourvus d'annotations et les annotations d'un même énoncé peuvent être séparées en plusieurs îlots disjoints.

Syntactic Annotations

Ex. of a formal model : Tree Adjoining Grammar (TAG)

En analyse automatique, les représentation syntaxiques combinent des éléments atomiques du formalisme syntaxique au moyen d'une ensemble d'opérateurs ou règles sur les structures syntaxiques pour construire l'arbre syntaxique qui correspond à un énoncé (approche de la grammaire générative). Par exemple le formalisme des grammaires d'arbres adjoints (Tree Adjoining Grammars) de A.K. Joshi, combine des arbres au moyens de 2 opérations :

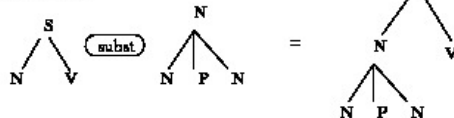
- ▶ la substitution (remplacement d'une feuille par un arbre dont la racine porte la même étiquette que la feuille qu'il remplace)
- ▶ l'adjonction remplacement d'un noeud interne de l'arbre par un arbre dont la racine porte la même étiquette que la feuille qu'il remplace et dont l'une des feuille porte la même étiquette que la racine de l'arbre.

Syntactic Annotations

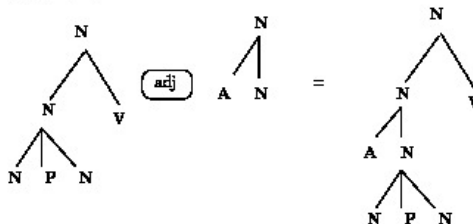
Ex. of a formal model : Tree Adjoining Grammar (TAG)

Tree Adjoining Grammar Operations

Substitution



Adjunction



Syntactic Annotations

Ex. of a formal model : Tree Adjoining Grammar (TAG)

En TAG la construction d'une analyse produit 2 résultats :

1. l'arbre dérivé, c'est-à-dire l'arbre syntaxique couvrant l'énoncé analysé
2. l'arbre de dérivation, l'arbre représentant les différentes opérations de substitution (traits pointillés) et d'adjonction (trait plein) qui ont servi à produire l'arbre dérivé à partir des arbres contenus dans la grammaire définie pour le langage analysé.

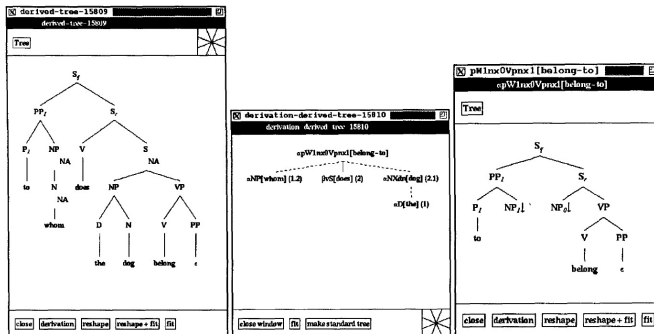


Figure 4: *left*, a derived tree, *middle*, its derivation, *right*, an elementary tree participating in the derivation.

Universal Dependencies

UD

<https://universaldependencies.org/>

<https://universaldependencies.org/guidelines.html>

<https://universaldependencies.org/u/overview/syntax.html>

<https://universaldependencies.org/fr/index.html>

<https://universaldependencies.org/fr/dep/index.html>



By contrast, compounds are annotated to show their modification structure, including a regular concept of head:



Analyse en dépendances

Sémantique, Constituants vs Dépendances, Structures

Grammaires de dépendance formelles et théorie Sens-Texte,
Sylvain Kahane, TALN, 2001

<https://kahane.fr/wp-content/uploads/2017/06/dg-taln2001.pdf>

Why to choose dependency rather than constituency for syntax : a formal point of view,
Sylvain Kahane, 2012

<https://kahane.fr/wp-content/uploads/2017/01/dependency-festchrift2012.pdf>

Sparse Logistic Regression with High-order Features for Automatic Grammar Rule Extraction from Treebanks,
Santiago Herrera, Caio Corro, Sylvain Kahane, 2024,

<https://arxiv.org/abs/2403.17534>

Stanza est actuellement une solution en accès libre et gratuit d'analyse syntaxique automatique en dépendances écrite en python. La boîte à outils a été développée par l'Université de Stanford ; elle permet de traiter une soixantaine de langues et elle offre d'excellentes performances en qualité d'annotation et en vitesse de traitement.

Pour installer stanza dans un environnement Ubuntu, vous pouvez l'installer directement avec pip en tapant dans un terminal

```
bash : pip install stanza
```

Alternativement, vous pouvez l'installer à partir des sources, dans ce cas il faut d'abord créer un répertoire pour recevoir le code source, par ex. `cd ; mkdir stanza ; cd stanza` puis d'exécuter la commande de téléchargement `git clone https://github.com/stanfordnlp/stanza.git` ou si vous n'avez pas installé git sur votre machine `(sudo apt-get install git)` de télécharger directement l'archive `https://github.com/stanfordnlp/stanza/archive/master.zip`.

Une fois votre terminal bash positionné dans le répertoire vous pouvez exécuter la commande (**attention à ne pas oublier le point après moins "e" espace**) : `pip install -e .`

Ensuite il suffit de lancer un interprète python et d'exécuter les commandes suivantes :

```
import stanza  
nlp = stanza.Pipeline('fr')
```

La première fois, l'exécution prends un peu de temps car stanza télécharge les ressources spécifiques à la langue sélectionnée (ici le français). Ensuite vous pouvez l'utiliser, par exemple :

```
a_text = "Il fait beau aujourd'hui."  
doc = nlp( a_text )  
print( doc.sentences[0].tokens )
```

```
from pprint import pprint # pour faire lisible et joli
import stanza
nlp = stanza.Pipeline('fr')
a_text = "Il fait beau aujourd'hui."
doc=nlp( a_text )
print( doc.sentences[0].tokens)

ts = 0
for s in doc.sentences:
    print( 'la phrase {0} est :'.format( ts ))
    for t in s.tokens:
        print( 'token {0} = {1}'.format( t.id, t.text ))
        print( 'indices des caractères de {0} (inclus) à {1} (exclu)'.format( t.start_char, t.end_char))
        print( '\t les informations associées aux mots correspondant au token : ')
        # pour certaines langues, comme le français, il peut y avoir parfois plusieurs mots associés à un token.
        for w in t.words:
            # pprint( w._dict_ ) # utiliser à fin de debug
            print( '\t\t mot {0} = {1}'.format( w.id, w.text ))
            print( '\t\t partie du discours = {0}'.format( w.upos ))
            print( '\t\t traits morpho-syntaxiques = {0}'.format( w.feats ))
            print( '\t\t lemme = {0}'.format( w.lemma ))
            print( '\t\t indice de la cible de la dépendance {0} partant de ce token = {1}'.format( t.words[0].deprel,
                                                    t.words[0].head ))

        print( "=====" )
    ts += 1
```



```
>>>
=====
2020-05-06 17:57:58 INFO: RESTART: /home/pap/src/stanza_stanford_nlp_python/stanza/quic
2020-05-06 17:57:58 INFO: Loading these models for language: fr (French):
=====
| Processor | Package |
|-----|-----|
| tokenize | gsd      |
| mw       | gsd      |
| pos      | gsd      |
| lemma    | gsd      |
| depparse | gsd      |
| ner      | wikiner   |
=====

2020-05-06 17:57:58 INFO: Use device: cpu
2020-05-06 17:57:58 INFO: Loading: tokenize
2020-05-06 17:57:58 INFO: Loading: mw
2020-05-06 17:57:58 INFO: Loading: pos
2020-05-06 17:57:59 INFO: Loading: lemma
2020-05-06 17:57:59 INFO: Loading: depparse
2020-05-06 17:58:00 INFO: Loading: ner
2020-05-06 17:58:01 INFO: Done loading processors!
[[
  {
    "id": "1",
    "text": "il",
    "lemma": "il",
    "upos": "PRON",
    "feats": "Gender=Masc|Number=Sing|Person=3|PronType=Prs",
    "head": 2,
    "deprel": "nsubj",
    "misc": "start_char=0|end_char=2"
  },
  {
    "id": "2",
    "text": "fait",
    "lemma": "faire",
    "upos": "VERB",
    "feats": "Mood=Ind|Number=Sing|Person=3|Tense=Pres|VerbForm=Fin",
    "head": 0,
    "deprel": "root",
    "misc": "start_char=3|end_char=7"
  },
  {
    "id": "3",
    "text": "beau",
    "lemma": "beau",
    "upos": "ADJ",
    "feats": "Gender=Masc|Number=Sing",
    "head": 2,
    "deprel": "xcomp",
  }
]]
```

stanza/Stanford Core NLP

- ▶ documentation sur les stanza pour produire les dépendances syntaxiques d'une phrase : <https://stanfordnlp.github.io/stanza/depparse.html#accessing-syntactic-dependency-information>
- ▶ visualization interactive en ligne des informations de dépendances de stanza : <https://corenlp.run/>
- ▶ un exemple de code python pour dessiner l'arbre de dépendances syntaxiques :

```
import nltk

from nltk.tokenize import word_tokenize*

from nltk.tag import pos_tag

doc = "He derives great joy and happiness from cycling"

doc = nltk.word_tokenize(doc)

doc = nltk.pos_tag(doc)

# identification des noms introduits par un déterminant (optionnel) et/ou
# une suite d'adjectifs ante-posés (optionnels).
grammar = "NP: {<DT>?<JJ>*<NN>}"

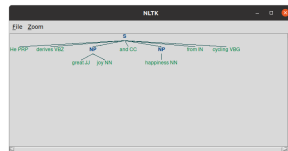
cp = nltk.RegexpParser(grammar)

# calcul de l'arbre de dépendances
result = cp.parse(doc)

# affichage de l'arbre de dépendances
result.draw()
```

Source : [https://](https://stackoverflow.com/questions/53312641/draw-dependency-tree-in-python-with-stanfordcorenlp-format)

<https://stackoverflow.com/questions/53312641/draw-dependency-tree-in-python-with-stanfordcorenlp-format>



- Notez qu'il existe spacy un analyseur syntaxique en dépendance aux performances comparables à celles de stanza.

```

spacy_demo.py - /home/pap/COURSES/inalco_22_23/L3/SEMESTER_2/course_S2_6_20230308/spa...
File Edit Format Run Options Window Help

# NOTE: installation de spacy depuis un terminal de commande (shell bash)
# bash> pip install spacy

import spacy

nlp = spacy.load("en_core_web_sm")

nlp = spacy.load("fr_core_news_sm")

doc = nlp("Today, the sun is shining.")

spacy.displacy.serve(doc, style="dep")

## Pour voir les dépendances, ouvrir
## votre navigateur web à l'URL: http://0.0.0.0:5000
# |-----
##
##Using the 'dep' visualizer
##Serving on http://0.0.0.0:5000 ...
##
##127.0.0.1 - - [08/Mar/2023 12:59:08] "GET / HTTP/1.1" 200 5072
##127.0.0.1 - - [08/Mar/2023 12:59:09] "GET /favicon.ico HTTP/1.1" 200 5072

```

